

①

Resumo 5

The importance of (test) automation:

- Software hoje é essencial e precisa ser entregue com rapidez e qualidade.
- Continuous delivery garante que o software esteja sempre pronto para produção.
- Testes manuais são lentos, repetitivos e sujeitos a erros.
- Automatizar testes encurta o ciclo de feedback, aumenta a confiança e facilita mudanças grandes no código.

The test pyramid:

- Conceito criado por Mike Cohn: três camadas principais - unit tests, service test, UI tests.
- Essência:
 - Muitos testes pequenos e rápidos (unitários)
 - Alguns testes de integração
 - Poucos testes de alto nível (UI/end-to-end)
- Evitar o "ice-cream cone" (muitos testes caros de UI e poucos unitários).
- Os nomes originais podem ser confusos, adapte-os ao contexto da sua equipe

Tools and Libraries

- JUnit: Framework de testes em Java
- Mockito: Mocks e stubs
- WireMock: Simular serviços externos
- Pact: Consumer-driven contract test (CDC)
- Selenium: Testes de interface web automatizados
- REST-assured: Testes de APIs REST.

Sample application

- Microserviço em Spring/Boot
- Endpoints: /hello, /hello/{lastname}, ~~/hello~~ weather
- Estrutura interna: controllers, repositories, clients, domain.
- Usa banco em memória (H2) para testes e Postgre SQL em produção

Unit tests:

- Base da pirâmide, mais numerosas
- Testam unidades pequenas (método, classe)
- Podem ser solitárias (todas as dependências substituídas por mocks) ou sociais (usam dependências reais simples)
- Estrutura: Arrange-Act-Assert (ou Given-when-then)
- Evitar testar métodos privados; se necessário, refatorar o teste

- Testar comportamento observável, não implementação interna

Integration tests:

- Garantem que a aplicação interage bem com dependências externas
- Exemplo:
 - Banco de dados (H2 para testes)
 - Serviços externos (simulador com WireMock)

- Testes mais lentos e mais caros que unitários
- Devem focar em pontos de integração (REST, DB, filas, filesystem)

Contracts tests (CDC)

- Importantes em arquiteturas de microsserviços
- Consumidor escreve expectativa (Post para arquivo JSON)
- Provedor valida o contrato contra esse arquivo.
- Benefícios:
 - Comunicação automatizada entre equipes
 - Evita que mudanças quebrem consumidores
 - Estimula autonomia das times

UI tests:

- Testam o comportamento da interface (Web, REST, API, CLI, etc)

- Podem ser unitários (em frameworks frontend) ou com ferramentas como Selenium
- Layout e usabilidade são mais difícil de testar automaticamente.
- Para design e UX, é preciso testar exploratórios e manuais

End-to-end tests:

- Testam o sistema completo, como o usuário o usa
- Exemplo: fluxo de compra em e-commerce
- Custos, lentos e frágeis (Flaky)
- Devem ser poucos e focados nos fluxos de maior valor
- Tecnologias: Selenium, Webdriver, REST-assured (para API's)

Acceptance tests:

- Validam se funcionalidades atendem à perspectiva do usuário
- Podem ser escritos em estilo BDD (Given-When-Then).
- Não precisam estar no topo da pirâmide: podem ser em como das mais baixas.
- Ex: Teste que confirma que um item foi adicionado ao carrinho

⑤ Exploratory Testing:

- Mesmo com automação, ainda há valor em teste manual criativo
- Testador tenta quebrar o sistema procurando bugs, problemas de design e ~~desempenho~~ desempenho
- Os bugs encontrados devem ser documentados e transformados em testes automatizados.

Confusion about Terminology:

- Termos com Unit, integration, component, service são usados de formas diferentes por equipes.
- O importante é definir e alinhar os Termos dentro do time/organização

Deployment pipeline:

- Estrutura dos testes na pipeline depende da velocidade e escopo, não do tipo de teste
- Testes rápidos → Fazer iniciais
- Testes lentos → Fazer posteriores
- EX: Alguns testes de integração rápidos podem rodar junto com os unitários.

⑥ Avoid Test duplication!

- Regra de ouro
 - Se só um teste de nível alto detecta erros, escrever também um de nível mais baixo
 - Coloque testes desnecessários tornam a suite lenta e difícil de manter
- Evitar redundâncias: testes desnecessários tornam a suite lenta e difícil de manter

Writing clean test code:

- Testes não são tão importantes quanto o código de produção
- Boas práticas:
 - Um caso por teste
 - Estrutura clara (Arrange, Act, Assert)
 - código legível > sucesso de reutilização
 - Refatorar com a Rule of Three

Conclusão:

- A pirâmide de testes ajuda a balancear velocidade, confiabilidade e cobertura
- Funciona para qualquer tipo de aplicação (microserviços, mobile, IoT, web)
- Investir em uma boa suite de testes traz paz, confiança e agilidade ao desenvolver